

CMDA-4654
Group Exercise 1B

Sophia Gavasheli and Jacob Carlson

October 05, 2026

1. Reintroduction

We are Sophia Gavasheli and Jacob Carlson, and we our group 17. Our chosen domain is the sports category.

2. Executive Summary and Research Questions

KNN Regression

We are using a KNN regression model to predict the runs a team scored in any given game. Runs will be predicted by the following variables.

Variable	Description
AB	At Bats
1b	Singles
2B	Doubles
3B	Triples
HR	Home Runs
SB	Stolen Bases
CS	Caught Stealing
BB	Base on Balls
SO	Strikeouts
HBP	Hit by Pitch
SH	Sacrifice Hits
SF	Sacrifice Flies

The data frame that these variables were taken from was not used in part A of our exercise, but is still part of the larger data set we pulled from. Using runs as a metric was an explored idea in part A, but was not used in our final report, so we decided to further explore it here, since it looked like it could have a relationship with the other variables like home runs and at bats.

KNN Classification and Naive Bayes

For our KNN Classification and Naive Bayes models, we chose to predict a player's position using the following offensive stats:

Variable	Description
BA	Batting Average
HRA	Home Runs Average
BBA	Base on Balls Average
SOA	Strikeout Average

We standardized this data to ensure KNN and Naive Bayes were using the same inputs. We actually spent a lot of our part A analysis on position metric, so exploring further with the two models was a natural part B.

In part A, we used the data frames `fieldingpost.csv` and `batting.csv`. Fielding-post only included post season stats(playoffs), and wasn't big enough for our models, so we switched to `fielding.csv`, which contains the entire seasons. We also used `Teams.csv` for our KNN regression, as it contained all of the data used in our model with minimal cleaning and merging between data sets.

3. Methodology & Modeling Setup

Data Preprocessing

All of our predictors were numerical, so there was no need to handle categorical leveling or do any smoothing to our data. We did a 70/30 training-test split since the data sets were very large, and we preferred to keep the majority in our training.

KNN regression

For our KNN regression, `Teams.csv` includes a great many more predictors than what was used. We built the data frame `team_clean`, which keeps only the American and National leagues, removes `NaN` rows, and takes in only our predictors, `TeamId`, `yearID`, and our prediction variable `Runs`. We also combined the HBP and BB predictors into one, as they provide the same outcome in a game. We also divided all of our predictors by games to establish the per game basis for our prediction.

We standardized our data set under the basic z-score scaling. As mentioned above, the predictor `HBPxBB` was a sum of two predictors, and was unfairly weighted against our other data. Furthermore, given the nature of the game, some of our predictors had a reported 0 weight on our distance, so standardizing was necessary.

The table below shows why scaling matters: for two team-seasons, the unscaled distance is dominated by a few large-valued predictors, while after z-scoring each predictor contributes a more balanced share.

Table 1: Share of squared Euclidean distance contributed by each predictor (first two games).

	X1B	X2B	X3B	HR	SO	HBPxBB	SB	SF	CS	AB
Raw (share of squared distance, pct)	8.2	0	0.0	0.4	48.6	42.1	0.7	0.0	0.0	0
Z-scored (share of squared distance, pct)	10.8	0	0.1	1.8	9.1	71.6	4.0	1.8	0.7	0

KNN and Naive Bayes Classification

For our KNN classification and Naive Bayes models, we first created `primary_pos`, which read through the player's data, found their most played position, and changed their other reported positions to the highest one. We then combined `primary_pos` with batting, based on their `playerId` and the `yearID`, so that we could compare the positions to their offensive stats. We also took only the American and National league players, following our analysis in part A.

We wanted to limit classification responses, so we combined the infield positions (1B,2B,3B, SS) into an `Infield` category.

In baseball, different positions get different amounts of playtime. Furthermore, because we combined four possible positions into one, our data was even more skewed, and in the case of KNN classification, our distances would be poorly weighted. We therefore decided to scale all of our predictors based on At bats, providing the predictors described above (BA, HRA, BBA, SOA). While the standardizing would have less affect on our Naive Bayes model, since our goal is to compare it with our classification, we chose to scale the predictors before running either model.

Table 2: Share of squared Euclidean distance contributed by each predictor (first two team-seasons).

	BA	HRA	BBA	SOA
Raw (share of squared distance, pct)	51.7	13	20.8	14.5
Z-scored (share of squared distance, pct)	40.3	44	13.0	2.8

4. Analysis, Tuning & Results

KNN Regression

K Tuning with Cross Validation

We searched $k = 1, \dots, 80$ using 10-fold CV on the training data only.

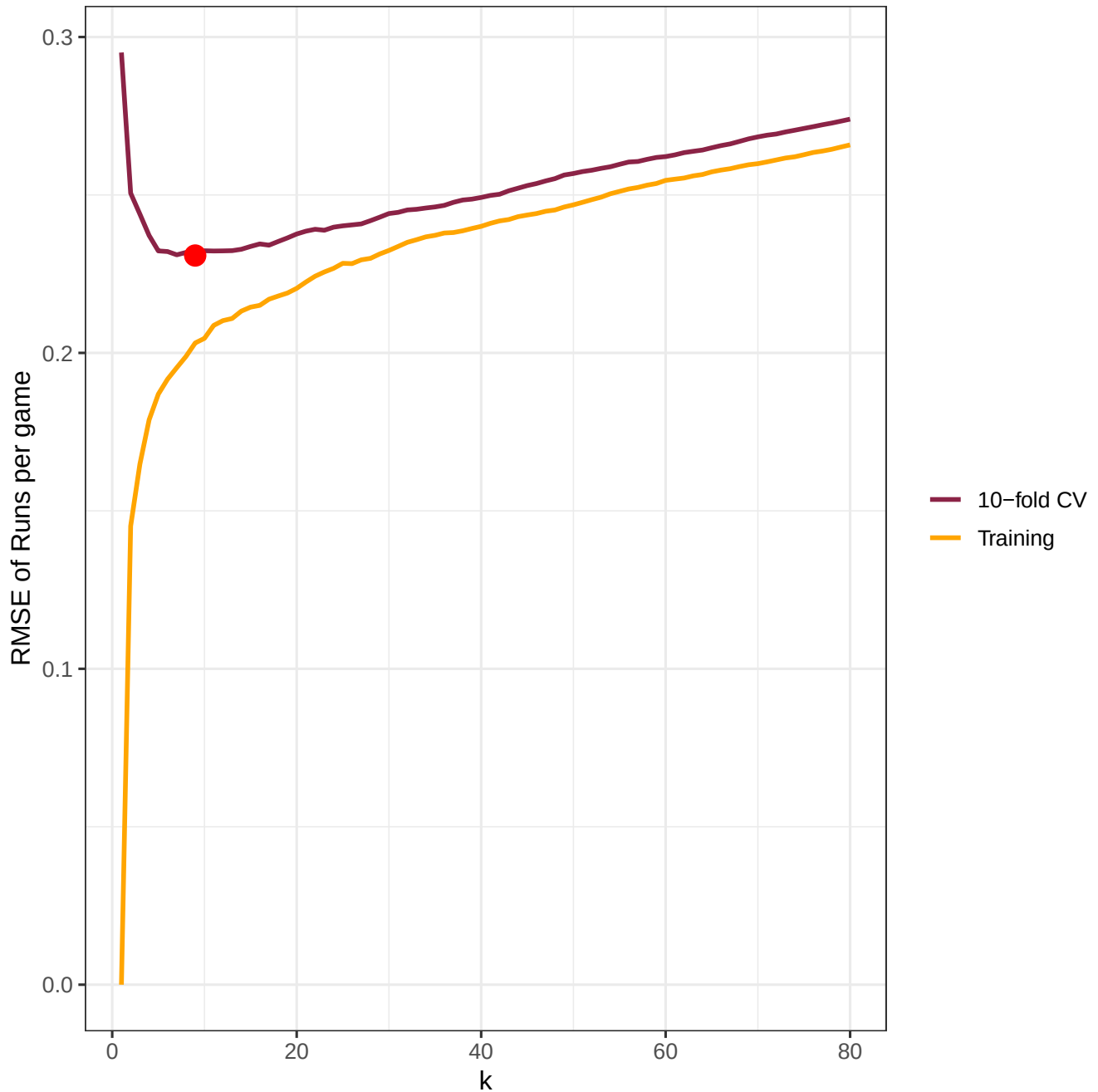


Figure 1: kNN regression tuning: cross-validated and training RMSE versus k . The red point marks the CV-optimal k .

The CV-optimal value is $k^* = 9$. Very small k gives a near-zero training RMSE (each point is nearly its own neighbor) but a worse CV RMSE, which is overfitting. As k becomes large the two curves rise together, which is underfitting.

Table 3: Top five candidate values of k by cross-validated RMSE.
The chosen optimum is $k = 9$.

k	CV RMSE	Training RMSE
9	0.231	0.203
7	0.231	0.195
8	0.232	0.199
6	0.232	0.192
11	0.232	0.209

Test-set performance versus the null model

Table 4: kNN regression on the held-out test set versus the null (mean-only) model.

Model	RMSE	MAE	R-squared
Null (training mean)	0.563	0.450	0.000
KNN ($k = 9$)	0.233	0.184	0.829

RMSE: Our null model was off by about half of a run every game when predicting the training average. Our $k = 9$ was off by about a quarter of a run every game, which is about a 50% reduction compared to our null model.

MAE: Our null models' absolute average miss is a little under half a run every game. The $k = 9$ model was off a little under half that, also being a little more than a 50% reduction compared to the null.

R Squared: As expected, our null model takes 0% of the predictors into account for its predictions. However, our $k = 9$ model takes 82%, signifying that the 10 predictors are a strong representation of the runs scored.

KNN Classification

We searched odd k from 1 to 51 with 10-fold CV on the training data only. Each candidate was scored with two metrics: balanced accuracy and log-loss. Log-loss is minimized at $k = 51$ and balanced accuracy is maximized at $k = 41$; the two metrics did not agree. We chose the final $k^* = 51$ using **log-loss**, because it is a strictly proper scoring rule that rewards well-calibrated probabilities and is not affected by the arbitrary argmax decision rule, which matters because we compare kNN to Naive Bayes on probability quality as well as on hard classification.

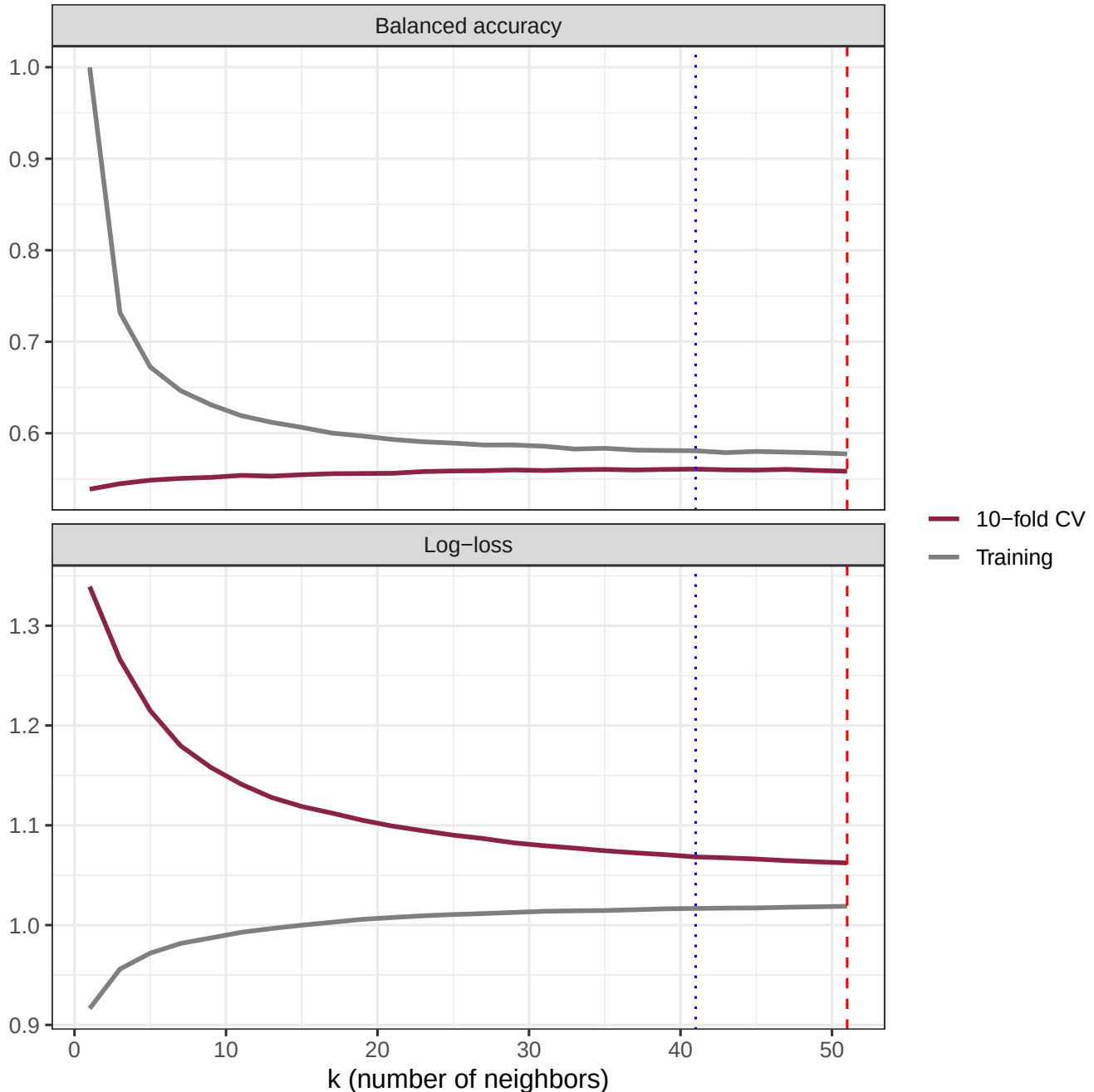


Figure 2: kNN classification tuning. Top: log-loss (lower is better). Bottom: macro balanced accuracy (higher is better). Red dashed line: best k chosen by CV log-loss. Blue dotted line: k with best CV balanced accuracy.

Table 5: Top five candidate values of k by cross-validated log-loss. Chosen optimum: $k = 51$.

k	CV log-loss	CV balanced acc.	CV macro-F1
51	1.062	0.558	0.348

k	CV log-loss	CV balanced acc.	CV macro-F1
49	1.063	0.559	0.350
47	1.065	0.561	0.352
45	1.066	0.560	0.351
43	1.067	0.560	0.352

Naive Bayes

Table 6: Table of Priors and Counts

	Catcher	Infield	Outfield	Pitcher
Count	5839.000	18331.000	15033.000	1167.000
Prior	0.145	0.454	0.372	0.029

This table shows the catcher and outfield positions being the most populated.

Table 7: Prediction Matrix of Positions for Naive Bayes

	Catcher	Infield	Outfield	Pitcher
Catcher	8	12	13	0
Infield	1278	4124	3012	215
Outfield	407	1278	1446	13
Pitcher	59	85	39	122

Table 8: Confusion Matrix of Positions for Naive Bayes

	Catcher	Infield	Outfield	Pitcher
Catcher	8	12	13	0
Infield	1278	4124	3012	215
Outfield	407	1278	1446	13
Pitcher	59	85	39	122

The prediction matrix for the null model follows the earlier count and prior, showing that Infield was the most populated position in the data set.

Table 9: Null Model Confusion Matrix of Positions for Naive Bayes

	Catcher	Infield	Outfield	Pitcher
Catcher	0	0	0	0
Infield	1752	5499	4510	350
Outfield	0	0	0	0
Pitcher	0	0	0	0

	Model	Accuracy	BalAcc	Kappa	LogLoss
1	Naive Bayes	0.4706465	0.5626264	0.08526979	1.068883
2	No-skill	0.4540500	0.5000000	0.00000000	1.108447

comparison between Naive Bayes and Null model

Comparing KNN vs Naive Bayes Classification

5. Discussion and Real World Implications

6. Citations and Data Link

Lahman Baseball Database – Society for American Baseball Research. (2025). Retrieved September 29, 2026, from <https://sabr.org/lahman-database/>

The data is linked [here](#).

7. Appendix

Data cleaning